The Bevi Ranking and Recommendation System

Mathematical specification of `@bevi/ranking` v1.2.0
and the extensions proposed on 2026-10-14

Bevi (Matt & Arjun)*

14 October 2026, revised 24 October 2026

Abstract

Bevi asks a user to compare a beverage ("Bev") against beverages they have already ranked and converts the answers into a personal 0–10 score, a global leaderboard and a prediction of how much a user would like a Bev they have not tried. This document states every formula the system uses, proves the properties the product relies on (termination and correctness of placement, exactness of stored scores, boundedness of predictions), explains the numbers users see (why two ratings of 7.6 and 2.0 display as 5.9), and specifies the extensions agreed in the 14 October 2026 session: category-aware comparisons over a weighted Bev graph (weights from taxonomy, content, text embeddings and flavour profiles, computed into Postgres and mirrored to Neo4j), the Taste Profile, Bev of the day and of the week, likes and an Explore ranking, a Smart Recommend filter for the map, nutrition normalisation and duplicate detection. It closes with the places where the SQL and TypeScript implementations currently differ.

Contents

1 Preliminaries and notation **1**

2 Pairwise placement **2**

3 Score from position **3**
3.1 Exact storage in hundredths 3

4 Statistics and normalisation **4**

5 The Bayesian leaderboard score **4**

6 Predicted score **5**
6.1 Content similarity between Bevs 5
6.2 Layer 1: affinity, content and global 6
6.3 Layer 2: user–user collaborative filtering 6
6.4 Blend, confidence and the displayed number 7

*Drafted from `packages/ranking/src`, `DEVELOPMENT-MATT.md` §8 and the second planning session (`CONVERSATION-2.md`). Sections 2–9 describe what is implemented and tested; Section 10 specifies the extensions, several of which have since shipped (its introduction says which). Revised on 2026-10-24 after the M-22 review of this paper against the package and the SQL; the review's correction lists are under M-22 in `TASKS.md`. Formula changes must bump `RANKING_VERSION` and regenerate the golden vectors in `shared/fixtures`.

7 The recommendation list **7**

8 Cold start **8**

9 Rank on Bevi and streaks **8**

10 Proposed extensions (session of 2026-10-14) **9**
10.1 Category-aware comparisons over a weighted Bev graph 9
10.2 Taste Profile 12
10.3 Bev of the day and Bev of the week 12
10.4 Likes and the Explore ranking 13
10.5 Smart Recommend on the map 13
10.6 Nutrition normalisation 13
10.7 Duplicate detection and merging 14
10.8 One ranking per Bev per user 14

11 Implementation notes, divergences and versioning **15**

A Constants **16**

B Symbols **18**

1 Preliminaries and notation

Let U be the set of users, B the set of Bevs and C the set of categories, with $\mathrm{cat} : B \to C$. A *ranking* is a triple $(u, b, s_{u,b})$ with a stored score $s_{u,b} \in [0, 10]$; each user ranks a Bev at most once (Section 10.8). Write $R(u) \subseteq B$ for the Bevs ranked by u, $R^{-1}(b) \subseteq U$ for the users who ranked b, and

$$n_u = |R(u)|, \qquad n_b = |R^{-1}(b)|, \qquad n_{u,c} = |\{b \in R(u) : \mathrm{cat}(b) = c\}|.$$

Every ranking belongs to one of three *buckets* chosen by the user's gut check, each mapped to a score band with integer edges $[\mathrm{lo}, \mathrm{hi}]$:

Bucket	Copy	Band	Database code
LOVED	"Loved it"	$[7, 10]$	3
FINE	"It was fine"	$[4, 7)$	2
DISLIKED	"Didn't like it"	$[0, 4)$	1

The bands are Beli's colour thresholds, so a score's colour and its bucket always agree. Within a bucket a user's Bevs form an ordered list, best first; position $i = 0$ is the best. All constants named in small capitals below are exported from `packages/ranking/src/constants.ts` and listed in Appendix A.

2 Pairwise placement

After the gut check selects a bucket, the new Bev b is placed into the user's ordered list $L = (\ell_0, \ldots, \ell_{n-1})$ for that bucket by binary search on "Which was better?" questions. The search is a pure state machine (`placement.ts`).

Definition 2.1 (Placement state)**.** A state is $(L, \mathrm{lo}, \mathrm{hi}, H)$ with $0 \le \mathrm{lo} \le \mathrm{hi} \le n$, where $[\mathrm{lo}, \mathrm{hi}]$ is the set of candidate *insertion indices* and H the history of answered questions. The initial state is $\mathrm{lo} = 0$, $\mathrm{hi} = n$. While $\mathrm{lo} < \mathrm{hi}$ the current question compares b with ℓ_m, $m = \lfloor (\mathrm{lo} + \mathrm{hi})/2 \rfloor$. The answers are

$$\begin{aligned} \textsc{new_better}: &\quad \mathrm{hi} \leftarrow m, \\ \textsc{existing_better}: &\quad \mathrm{lo} \leftarrow m + 1, \\ \textsc{tie}\ (\text{“Too close to call”}): &\quad \mathrm{lo} \leftarrow m + 1,\ \mathrm{hi} \leftarrow m + 1. \end{aligned}$$

Placement is *done* when $\mathrm{lo} = \mathrm{hi}$; the result is insertIndex $= \mathrm{lo}$. Undo pops H and restores the previous $(\mathrm{lo}, \mathrm{hi})$. An empty list is done immediately at index 0.

The number of candidate indices is $c = \mathrm{hi} - \mathrm{lo} + 1$, initially $n + 1$.

Lemma 2.2 (Halving)**.** *Each non-tie answer maps c candidates to at most $\lceil c/2 \rceil$.*

Proof. Let $d = \mathrm{hi} - \mathrm{lo} = c - 1$ and $m = \mathrm{lo} + \lfloor d/2 \rfloor$. After NEW_BETTER the new count is $m - \mathrm{lo} + 1 = \lfloor d/2 \rfloor + 1 = \lfloor (c+1)/2 \rfloor = \lceil c/2 \rceil$. After EXISTING_BETTER it is $\mathrm{hi} - m = d - \lfloor d/2 \rfloor = \lceil d/2 \rceil = \lceil (c-1)/2 \rceil = \lfloor c/2 \rfloor \le \lceil c/2 \rceil$. □

Theorem 2.3 (Termination)**.** *Placement into a list of n Bevs asks at most $\lceil \log_2(n+1) \rceil$ questions. A tie ends the search at once.*

Proof. By Lemma 2.2, after k answers $c \le \lceil (n+1)/2^k \rceil$, which equals 1 as soon as $2^k \ge n + 1$. The code's `maxComparisons(n)` returns the bit length of n, $32 - \mathrm{clz}_{32}(n) = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$ for $n \ge 1$ (and 0 for $n = 0$). Examples: $n = 100 \Rightarrow 7$, $n = 1000 \Rightarrow 10$. □

Definition 2.4 (Consistent oracle)**.** A user is *consistent* with true slot $t \in \{0, \ldots, n\}$ if they answer NEW_BETTER against ℓ_m exactly when $m \ge t$ and EXISTING_BETTER exactly when $m < t$.

Theorem 2.5 (Correctness)**.** *Against a consistent oracle with true slot t, placement returns* insertIndex $= t$.

Proof. Invariant: $\mathrm{lo} \le t \le \mathrm{hi}$. It holds initially. At a question with midpoint m, if $m \ge t$ the oracle answers NEW_BETTER and $\mathrm{hi} \leftarrow m \ge t$; otherwise $m < t$, the answer is EXISTING_BETTER and $\mathrm{lo} \leftarrow m + 1 \le t$. The invariant survives and at termination $\mathrm{lo} = \mathrm{hi} = t$. □

Proposition 2.6 (Log consistency)**.** *Every logged comparison agrees with the final index i^*: a* NEW_BETTER *answer against ℓ_m has $i^* \le m$ (the new Bev ends above ℓ_m), an* EXISTING_BETTER *answer has $i^* \ge m + 1$, and a tie against ℓ_m has $i^* = m + 1$ (directly below the incumbent, which keeps precedence because it was ranked first).*

Proof. Each answer sets $\mathrm{hi} \le m$, respectively $\mathrm{lo} \ge m + 1$, and the interval only shrinks afterwards, so the final $i^* \in [\mathrm{lo}, \mathrm{hi}]$ satisfies the inequality. A tie sets both ends to $m + 1$. □

`submit_ranking` receives the index, the length of the list that was searched (`expectedBucketSize`; a mismatch is a `conflict`, and the app restarts on a fresh list) and every comparison as a (winner, loser, tie) triple; for a tie the incumbent is recorded as the winner and the flag is set. The server checks the index range and the list length and stores the log; it does not yet re-check the log against the index (Implementation note 11.5). A re-rank removes the Bev from its list first, chooses the bucket again and runs the same search.

3 Score from position

Definition 3.1 (Score)**.** In a bucket with band $[\mathrm{lo}, \mathrm{hi}]$, width $w = \mathrm{hi} - \mathrm{lo}$, holding $n \geq 1$ Bevs after insertion, the Bev at position $i \in \{0, \ldots, n-1\}$ has

$$\sigma(i, n) = \mathrm{hi} - w\,\frac{i+1}{n+1}. \tag{1}$$

Proposition 3.2. *For all $0 \leq i < n$: (i) $\mathrm{lo} < \sigma(i,n) < \mathrm{hi}$; (ii) σ is strictly decreasing in i; (iii) $\sigma(0,1)$ is the band midpoint (8.5, 5.5, 2.0); (iv) $\sigma(i,n) + \sigma(n-1-i,n) = \mathrm{lo} + \mathrm{hi}$ (the spacing is symmetric); (v) ordering a user's whole list by score equals ordering by (bucket, position).*

Proof. (i) follows from $0 < (i+1)/(n+1) < 1$; (ii) and (iii) are immediate; (iv) from $(i+1)+(n-i) = n+1$; (v) from (i), since the bands are disjoint and ordered. □

Inserting or removing a Bev changes n, so every score in that bucket is recomputed ("re-spacing"). The spacing $(i+1)/(n+1)$ was preferred to the midpoint rule $(i+\frac{1}{2})/n$ because it is symmetric under re-indexing and trivial to write in SQL.

3.1 Exact storage in hundredths

Scores are stored as `numeric(4,2)`. To guarantee that TypeScript and Postgres agree on every half-cent, the stored value is computed in integer arithmetic (`storedScoreCents`).

Proposition 3.3 (Exact cents)**.** *Let $D = n+1$ and $N = 100\,(\mathrm{hi}\,D - w\,(i+1))$, both integers. Then $100\,\sigma(i,n) = N/D$ and the half-up rounding of $100\,\sigma$ to an integer is*

$$\mathrm{cents}(i,n) = \left\lfloor \frac{2N + D}{2D} \right\rfloor,$$

which both engines compute exactly. For FINE *and* DISLIKED *the result is capped at $100\,\mathrm{hi} - 1$ so the open upper edge is respected; the cap is first reached at $n = 599$ (*FINE*) and $n = 799$ (*DISLIKED*).*

Proof. $100\sigma = 100\,\mathrm{hi} - 100\,w(i+1)/D = N/D$. Rounding half up is $\lfloor x + \frac{1}{2} \rfloor$ with $x = N/D$, i.e. $\lfloor (2N+D)/(2D) \rfloor$. All quantities are integers below 2^{53}, so IEEE doubles and Postgres integer division are exact. The top position rounds up to hi exactly when $100w/(n+1) \leq \frac{1}{2}$, i.e. $n+1 \geq 200w$: $n \geq 599$ for $w = 3$ and $n \geq 799$ for $w = 4$. □

Example 3.4 (Reading a stored score)**.** A stored 7.60 in LOVED means $\frac{i+1}{n+1} = 0.8$ up to the half-cent rounding: the Bev is 4th of 4, 8th of 9, 12th of 14, 16th of 19, … (position $4k$ in a bucket of $5k-1$; no other (i,n) with $n \leq 60$ rounds to 7.60). A stored 2.00 in DISLIKED is the middle of an odd-sized bucket: a bucket of one, 2nd of 3, 3rd of 5, and so on. Both numbers from the session are therefore ordinary positions, not anomalies.

4 Statistics and normalisation

All standard deviations are *population* deviations ($\sqrt{\sum (s-\mu)^2/n}$, SQL `stddev_pop`): one rating gives 0 rather than `NULL`, and no inference is ever done on them.

Definition 4.1 (Per-Bev and per-user statistics)**.** For a Bev b: n_b, $\mu_b = \frac{1}{n_b} \sum_{v \in R^{-1}(b)} s_{v,b}$, σ_b. For a user u overall: n_u, μ_u, σ_u over all their stored scores; per category c: $n_{u,c}, \mu_{u,c}, \sigma_{u,c}$.

Definition 4.2 (Effective statistics)**.** With SIGMA_FLOOR $= 1$, MIN_N_USER $= 3$ and the global statistics (μ_G, σ_G) over all stored scores (falling back to $(5, 2.5)$ only when the system has no rankings),

$$(\mu_u^*, \sigma_u^*) = \begin{cases} (\mu_u, \max(\sigma_u, 1)) & n_u \geq 3, \\ (\mu_G, \max(\sigma_G, 1)) & \text{otherwise.} \end{cases}$$

Definition 4.3 (z-score)**.** $z_{u,b} = (s_{u,b} - \mu_u^*)/\sigma_u^*$, always against the user's *overall* effective statistics.

The floor keeps z finite and stops a user whose first three scores are nearly identical from producing enormous z-scores.

Proposition 4.4 (Why not per-category z-scores)**.** *If z-scores were computed against the per-category mean $\mu_{u,c}$, then $\sum_{b \in R(u),\, \mathrm{cat}(b)=c} z_{u,b} = 0$ for every u and c, and every category affinity would be identically zero.*

Proof. $\sum_b (s_{u,b} - \mu_{u,c}) = n_{u,c}\,\mu_{u,c} - n_{u,c}\,\mu_{u,c} = 0$. □

Definition 4.5 (Category affinity)**.** With K_AFF $= 2$,

$$\mathrm{aff}_{u,c} = \frac{\sum_{b \in R(u),\ \mathrm{cat}(b)=c} z_{u,b}}{n_{u,c} + \text{K_AFF}}, \qquad \mathrm{aff}_{u,c} = 0 \text{ when } n_{u,c} = 0. \tag{2}$$

Definition 4.6 (Normalised global score of a Bev)**.** With K_G $= 3$,

$$g_b = \frac{\sum_{v \in R^{-1}(b)} z_{v,b}}{n_b + \text{K_G}}, \qquad g_b = 0 \text{ for a new Bev.} \tag{3}$$

Proposition 4.7 (Shrinkage as a posterior mean)**.** *If $z_1, \ldots, z_n \sim \mathcal{N}(\theta, 1)$ independently and $\theta \sim \mathcal{N}(0, 1/K)$, the posterior mean of θ is $\sum z_i/(n+K)$. Hence (2) and (3) are Bayesian estimates of a user's mean preference for a category, and of a Bev's mean normalised quality, under a neutral prior with the weight of K observations.*

Proof. The posterior precision is $n + K$ and the precision-weighted mean is $(n\bar{z} + K \cdot 0)/(n+K)$. □

5 The Bayesian leaderboard score

The number shown on a Bev everywhere in the app (leaderboard, search results, feed cards, Bev detail) is not its plain mean.

Definition 5.1 (Bayesian average)**.** With prior mean m and prior weight $C = \text{BAYES_C} = 3$,

$$\mathrm{bayes}_b = \frac{C\,m + n_b\,\mu_b}{C + n_b} = \frac{C\,m + \sum_v s_{v,b}}{C + n_b}, \qquad \mathrm{bayes}_b = m \text{ when } n_b = 0. \tag{4}$$

Proposition 5.2. *(i)* bayes_b *lies between m and μ_b; (ii) it is the convex combination $\lambda_n \mu_b + (1-\lambda_n)m$ with $\lambda_n = n_b/(n_b+C)$, so the Bev's own ratings carry weight $25\%, 40\%, 62.5\%, 77\%$ at $n_b = 1, 2, 5, 10$; (iii)* $\mathrm{bayes}_b \to \mu_b$ *as $n_b \to \infty$; (iv) it is the posterior mean of θ_b when $s_{v,b} \sim \mathcal{N}(\theta_b, \sigma^2)$ and $\theta_b \sim \mathcal{N}(m, \tau^2)$ with $C = \sigma^2/\tau^2$.*

Proof. (i)–(iii) are algebra on (4). (iv): the posterior precision is $1/\tau^2 + n/\sigma^2$ and the posterior mean $(m/\tau^2 + n\mu_b/\sigma^2)/(1/\tau^2 + n/\sigma^2)$; multiply numerator and denominator by σ^2. □

Example 5.3 (The 5.9 from the session)**.** Two users ranked the same Bev at 7.6 and 2.0. The plain mean is 4.8; the app showed 5.9. By (4),

$$5.9 = \frac{3\,m + 7.6 + 2.0}{3 + 2} \iff m = \frac{5 \times 5.9 - 9.6}{3} \approx 6.63,$$

so the global mean of all stored scores was about 6.6 at that moment (any $m \in [6.55, 6.71\overline{6})$ gives bayes $\in [5.85, 5.95)$, which displays as 5.9). The prior, worth three ratings, outweighs the two real ones, which is exactly the design: a single 10/10 cannot top the chart ($(18 + 10)/4 = 7.0$ with $m = 6$). **Nobody's leaderboard rank enters the formula**; the only inputs are the Bev's own scores, their count, and the prior. The leaderboard, the feed and the Bev-of-the-period card show the live figure rounded to one decimal; `bev_stats.bayes_score`, which the search results and the Bev page read, is stored to two decimals first, so the two paths can differ in the displayed digit at a boundary such as 5.945 (Implementation note 11.6).

Prior mean. The mean of all stored scores in the Bev's category once that category has at least BAYES_CATEGORY_PRIOR_MIN $= 30$ rankings, else the global mean; one m per board. The overall board, the feed cards, the Bev of the period and the stored per-Bev figure all use the global m (`bayesian.ts`; SQL `bayes_prior`, since 20261019000200). With no rankings in the system $m = 5$. See Implementation note 11.2 for the functions that still differ.

Eligibility and order. A Bev joins a leaderboard once $n_b \geq$ MIN_RATERS $= 2$; below that it belongs in a "New" strip. Order: bayes_b descending, then n_b descending, then name ascending in code-point order (SQL `collate "C"`), then id, so the order is total and identical in both engines (`get_bev_leaderboard` since 20261019000200; for the one remaining difference see Implementation note 11.3).

Tuning rule. C should be roughly the number of ratings at which a Bev's own mean deserves as much trust as the prior; raise C to 5 when the median n_b over rated Bevs passes 10.

6 Predicted score

The prediction $\hat{s}_{u,b}$ ("You'd rate it ∼8.2") is computed in z-units and mapped back through the user's effective statistics. It blends a content layer and a collaborative layer; every function reads one immutable `PredictionSnapshot` (the caller's rankings, every user's statistics, every visible Bev's features and all rankings), so the app, the tests and any future server job give identical output.

6.1 Content similarity between Bevs

Definition 6.1 (Flavour tokens and brand key)**.** tok(b) is the set of lowercase words of flavour and name (apostrophes dropped), minus a stop list (*original, classic, drink, can, bottle* and unit words such as *oz, ml, pack, mini, regular, large, small*; `FLAVOR_STOPWORDS`) and size tokens (*12oz*, *500ml*, *2l* and bare numbers; `FLAVOR_SIZE_PATTERN`). brand(b) is the lowercase, whitespace-collapsed brand, or $\perp$ when empty.

Definition 6.2 (Content similarity)**.** With price tiers $\text{tier}(b) \in \{1, 2, 3, 4\}$ and Jaccard index $\text{J}(X, Y) = |X \cap Y|/|X \cup Y|$ (0 when both are empty),

$$\begin{aligned}\text{sim}(b, b') = 0.40\,\mathbf{1}\big[\text{cat}(b) = \text{cat}(b')\big] + 0.30\,\mathbf{1}\big[\text{brand}(b) = \text{brand}(b') \neq \perp\big] \\ + 0.20\ \text{J}(\text{tok}(b), \text{tok}(b')) + 0.10\Big(1 - \tfrac{|\text{tier}(b) - \text{tier}(b')|}{3}\Big).\end{aligned} \tag{5}$$

Pairs with sim $<$ SIM_MIN $= 0.1$ are dropped from every sum below.

Proposition 6.3. sim *is symmetric and takes values in* $[0, 1]$*, with* $\mathrm{sim}(b, b) = 1$ *whenever* b *has a brand and at least one flavour token (*$\mathrm{J}(\emptyset, \emptyset) = 0$ *by convention, so a branded Bev whose name and flavour are all stop words or sizes has* $\mathrm{sim}(b, b) = 0.8$*).*

Proof. Each of the four terms lies in $[0, 1]$ and the weights sum to one. □

6.2 Layer 1: affinity, content and global

Definition 6.4 (Layer 1)**.** For user u and candidate b,

$$\mathrm{content}_{u,b} = \frac{\sum_{b' \in R(u) \setminus \{b\}} \mathrm{sim}(b, b')\, z_{u,b'}}{\sum_{b' \in R(u) \setminus \{b\}} \mathrm{sim}(b, b') + 1}, \tag{6}$$

$$\hat{z}^{(1)}_{u,b} = 0.40\ \mathrm{aff}_{u,\mathrm{cat}(b)} + 0.40\, \mathrm{content}_{u,b} + 0.20\, g_b, \tag{7}$$

$$\mathrm{conf}^{(1)}_{u,b} = \min\Big(1, \frac{n_u}{10}\Big) \cdot \min\Big(1, \frac{n_{u,\mathrm{cat}(b)} + \sum_{b'} \mathrm{sim}(b, b')}{4}\Big). \tag{8}$$

The $+1$ in (6) shrinks the content term toward 0 when little similarity mass exists.

Proposition 6.5. $\hat{z}^{(1)}$ *is in z-units (its weights sum to one and each term is a shrunk mean of z-scores), and* $\mathrm{conf}^{(1)} \in [0, 1]$.

6.3 Layer 2: user–user collaborative filtering

User–user filtering was chosen for launch because with about twenty users every pair shares the onboarding starter Bevs, whereas item–item similarity needs co-raters per item pair, which a 380-Bev catalogue and twenty users cannot supply. Switch to item–item at roughly 200 users or when the median number of co-raters per rated item pair reaches 3.

Definition 6.6 (User similarity)**.** Let $\mathrm{common}(u, v) = R(u) \cap R(v)$ and $z_u[\cdot]$ the vector of u's z-scores on it. With CF_MIN_COMMON $= 2$ and CF_SIGNIFICANCE $= 5$,

$$\mathrm{sim}'(u, v) = \begin{cases} \cos\big(z_u[\mathrm{common}], z_v[\mathrm{common}]\big) \cdot \dfrac{\min(|\mathrm{common}(u, v)|, 5)}{5} & |\mathrm{common}(u, v)| \geq 2,\ u \neq v, \\ 0 & \text{otherwise,} \end{cases} \tag{9}$$

where $\cos(x, y) = x \cdot y / (\|x\| \|y\|)$ and is 0 if either vector is zero.

The factor $\min(|\mathrm{common}|, 5)/5$ is significance weighting: two users who agree on two Bevs are not yet neighbours of full weight.

Definition 6.7 (Layer 2)**.** Let $N_K(u, b)$ be the $K =$ CF_K $= 10$ users $v \neq u$ who ranked b with the largest $\mathrm{sim}'(u, v) > 0$ (ties by user id). Then

$$\hat{z}^{(2)}_{u,b} = \frac{\sum_{v \in N_K} \mathrm{sim}'(u, v)\, z_{v,b}}{\sum_{v \in N_K} \mathrm{sim}'(u, v)}, \qquad \mathrm{supp}_{u,b} = \sum_{v \in N_K} \mathrm{sim}'(u, v), \tag{10}$$

with $\hat{z}^{(2)} = 0$ and supp $= 0$ when no neighbour ranked b.

Proposition 6.8. *$\hat{z}^{(2)}_{u,b}$ is a convex combination of the neighbours' z-scores, hence* $\min_{v\in N_K} z_{v,b} \le \hat{z}^{(2)}_{u,b} \le \max_{v\in N_K} z_{v,b}$, *and* $0 \le \mathrm{supp}_{u,b} \le K = 10$.

Proof. The weights $\mathrm{sim}'(u,v)/\,\mathrm{supp}$ are non-negative and sum to one; each $\mathrm{sim}' \le 1$ and there are at most K of them. □

In score units this is the classical "user's mean plus the weighted average of similar users' deviations": $\mu^*_u + \sigma^*_u\,\hat{z}^{(2)}_{u,b}$.

6.4 Blend, confidence and the displayed number

Definition 6.9 (Blend)**.** With $\Lambda_{\max} = \text{LAMBDA_MAX} = 0.7$ and $\text{LAMBDA_USER_N} = 15$,

$$\lambda_{u,b} = \Lambda_{\max}\,\frac{\mathrm{supp}_{u,b}}{\mathrm{supp}_{u,b}+1}\,\min\Big(1, \frac{n_u}{15}\Big), \tag{11}$$

$$\hat{z}_{u,b} = (1-\lambda_{u,b})\,\hat{z}^{(1)}_{u,b} + \lambda_{u,b}\,\hat{z}^{(2)}_{u,b}, \tag{12}$$

$$\hat{s}_{u,b} = \mathrm{clamp}\big(\mu^*_u + \sigma^*_u\,\hat{z}_{u,b},\ 0,\ 10\big), \tag{13}$$

$$\mathrm{conf}_{u,b} = \max\big(\mathrm{conf}^{(1)}_{u,b},\ \min(1, \mathrm{supp}_{u,b})\big). \tag{14}$$

Proposition 6.10. $0 \le \lambda_{u,b} \le \Lambda_{\max}\cdot\frac{10}{11} < 0.64$*; with* $\mathrm{supp} = 0$ *the prediction is pure Layer 1, so the Phase 2 (content only) and Phase 3 (collaborative) products share one code path.*

Proof. $\mathrm{supp} \le 10$ by Proposition 6.8 and $x/(x+1)$ is increasing. □

The app displays $\hat{s}$ to one decimal with a tilde prefix and replaces the number by a lock when $\mathrm{conf} < \text{CONFIDENCE_LOCK} = 0.25$. Predictions are withheld entirely while $n_u < 3$ (the recommendations list is empty and the Bev detail shows "Rank 3 Bevs to see your prediction"). A Bev the user has ranked shows their own score instead.

7 The recommendation list

`recommend(snapshot, u, {n, seed})` returns the top n untried Bevs ($\text{RECOMMEND_N} = 20$ by default, capped at 50 by the API).

Definition 7.1 (Candidates and ordering)**.** Candidates are the Bevs in the snapshot not in $R(u)$, not excluded, and not alcoholic unless the user may see alcohol, sorted by id. Each candidate receives a prediction and an *ordering-only* jitter

$$\tilde{z}_b = \hat{z}_{u,b} + \varepsilon_b, \qquad \varepsilon_b \sim \mathcal{N}(0, \text{JITTER_SD}^2),\ \text{JITTER_SD} = 0.15,$$

drawn in id order from a seeded generator (FNV-1a hash of the seed into `mulberry32`; Gaussians by Box–Muller, $\varepsilon = \mathrm{sd}\sqrt{-2\ln(1-U_1)}\cos(2\pi U_2)$). The seed is `userId:localDay`, so a day's list is stable and the next day's shuffles slightly. The displayed score is the un-jittered $\hat{s}$.

Definition 7.2 (Exploration slots)**.** Let $\epsilon = \text{EPSILON} = 0.15$ and let $L = \min(n, |\text{candidates}|)$ be the list length. The list reserves $s = \min(\mathrm{round}(\epsilon n), L)$ slots at the positions

$$p_k = \left\lfloor \frac{(k+1)(L+1)}{s+1} \right\rfloor - 1, \qquad k = 0, \dots, s-1. \tag{15}$$

For $n = L = 20$: $s = 3$ and $(p_k) = (4, 9, 14)$.

Proposition 7.3. *The slot positions are distinct and lie in* $\{0, \ldots, L-1\}$ *whenever* $s \le L$.

Proof. $p_{s-1} = \lfloor s(L+1)/(s+1) \rfloor - 1 \le L-1$ since $s(L+1)/(s+1) < L+1$; $p_0 \ge \lfloor (L+1)/(s+1) \rfloor - 1 \ge 0$; consecutive positions differ by at least $\lfloor (L+1)/(s+1) \rfloor \ge 1$. □

Definition 7.4 (Exploration pool and draw)**.** Let med be the median of $\{\text{aff}_{u,c} : n_{u,c} > 0\}$ (0 if the user has no rankings). The pool is every candidate b with $g_b \ge 0$ whose category the user has never ranked or has $\text{aff}_{u,\text{cat}(b)} \le \text{med}$. Slots are filled by weighted sampling without replacement with weights $\exp(g_b)$ (a Boltzmann distribution over global quality at unit temperature), using the same generator after the jitter draws. Picks are tagged `isExploration` ("Something different"). Unfilled slots fall back to normal picks; the remaining positions take the non-exploration candidates in decreasing $\tilde{z}$.

Definition 7.5 (Reason)**.** Each recommendation names the largest contribution to (12): CATEGORY $= (1-\lambda)\,0.4$ aff, SIMILAR $= (1-\lambda)\,0.4$ content, NEIGHBOURS $= \lambda\,\hat{z}^{(2)}$, POPULAR $= (1-\lambda)\,0.2\,g_b$, ties in that order; all zero gives POPULAR.

8 Cold start

New user. Onboarding asks for $k = 10$ starter Bevs: a *core* of the 6 most-rated starters (rating count descending, then name, then id, skipping a starter whose category already has 2 core picks; the same for everyone, which maximises $|\text{common}(u, v)|$ for Layer 2), then 4 drawn with a generator seeded by the user id (so leaving and returning shows the same set), at most 2 per category over the whole set, aiming for at least 5 distinct categories, alcoholic categories excluded under 21. If the per-category cap makes k unreachable it is relaxed. Until the user has 3 rankings, their effective statistics are the global ones (Definition 4.2).

New Bev. No special code: $g_b = 0$ and supp $= 0$, so the prediction is affinity plus content only, and the Bev stays off the leaderboard until $n_b \ge 2$.

9 Rank on Bevi and streaks

Definition 9.1 (Rank points)**.** $\text{points}_u = 1 \cdot \text{bevsRanked}_u + 3 \cdot \text{currentStreak}_u + 2 \cdot \text{bevsCreated}_u$, where bevsCreated_u counts Bevs u added that at least one *other* user has ranked. Followers are deliberately excluded (gameable, unrelated to ranking). Users with no rankings are "Unranked" and do not count toward anyone's rank. Competition ranking: $\text{rank}_u = 1 + |\{v : \text{points}_v > \text{points}_u\}|$; display order is points, then Bevs ranked, then account age, then user id, so the order is total (SQL: `user_rank_points` and `get_user_leaderboard`, with `rank() over (order by points desc)`).

Definition 9.2 (Streak)**.** The day key of a ranking is the calendar date of its timestamp in the user's profile time zone. Given the set D_u of day keys with at least one ranking and today's key t, the current streak starts at t if $t \in D_u$, else at $t-1$ if $t-1 \in D_u$ (alive but not yet extended today), else is 0, and counts consecutive days backwards; the longest streak is the longest consecutive run in D_u. Several rankings on one day count once; re-ranks count (the re-rank day joins D_u and the original day stays); removals never break a streak. D_u is therefore the set of days with a ranking *event*, not the days of the rows that currently exist (Implementation note 11.4). A weekly variant (ISO weeks) is a constant change.

10 Proposed extensions (session of 2026-10-14)

Status on 2026-10-24. Shipped: the edge weights and the graph pipeline of Section 10.1 (`edges.ts`, `RANKING_VERSION` 1.2.0; `bev_edges` is live on prod), the Taste Profile data of Section 10.2 (A-11, #101), the Bev of the day and week of Section 10.3 (A-05, #96), likes and the Explore ranking of Section 10.4 (A-02 #70, A-04 #113) and the one-ranking rule of Section 10.8. Not built: anchored placement (M-21; waits on the taxonomy, D-04 and A-13), Smart Recommend (Section 10.5, parked), nutrition (Section 10.6) and the duplicate score and merge of Section 10.7 (the S-13 plan). Each subsection gives the definition precisely enough to code and test; where a choice was open it now records the decision.

10.1 Category-aware comparisons over a weighted Bev graph

Motivation. External feedback: ranking a Bev against a Bev of a very different kind feels wrong ("I'd always take mid Indian food over top-tier American food"). The agreed answer: *the first comparisons are against the most similar Bevs the user has ranked*, same subcategory first, then adjacent categories; categories become hierarchical and Bevs live in a graph whose edges carry similarity weights.

Taxonomy. Let T be a rooted tree of categories; today's 15 categories become internal nodes and Bevs point to leaves (e.g. coffee → hot, iced → mocha, matcha, …). Write $\mathrm{depth}(x)$ for the depth of a node and $\mathrm{lca}(x, y)$ for the lowest common ancestor. Depth is measured from an implicit root above the top-level categories: the root has depth 0 and a top-level category depth 1 (`treeSimilarity`, and the `treeDepth` line of the vectors' header). Two top-level categories therefore meet only at the root and have $\mathrm{sim}_T = 0$, siblings under one parent have $\mathrm{sim}_T = 2 \cdot 1/(2+2) = 0.5$, and with today's flat taxonomy (no `categories.parent_id` until A-13) sim_T is exactly the same-category indicator of (5).

Definition 10.1 (Tree similarity, Wu–Palmer)**.**

$$\mathrm{sim}_T(b, b') = \frac{2\ \mathrm{depth}(\mathrm{lca}(\mathrm{cat}(b), \mathrm{cat}(b')))}{\mathrm{depth}(\mathrm{cat}(b)) + \mathrm{depth}(\mathrm{cat}(b'))} \in [0, 1], \tag{16}$$

equal to 1 for the same leaf and decreasing as the common ancestor moves toward the root.

Definition 10.2 (Embedding and profile similarity)**.** Let $e_b \in \mathbb{R}^{384}$ be the unit-norm text embedding of "brand, name, flavour, category" (the `gte-small` model that ships with Supabase Edge Functions) and $\mathrm{sim}_{\mathrm{emb}}(b, b') = \max(0, e_b \cdot e_{b'})$. Let every Bev carry a *flavour profile* $\pi_b = (\mathrm{base}, \mathrm{carb}, \mathrm{caff}, \mathrm{sweet}, \mathrm{dairy}, \mathrm{alc})$: a categorical base family (cola, citrus, coffee, matcha, …), carbonation, dairy and alcohol as binary flags, caffeine and sweetness on an ordinal 0..3 scale. Then

$$\begin{aligned} \mathrm{sim}_{\mathrm{prof}}(b, b') = {} & 0.40\,\mathbf{1}[\mathrm{base}_b = \mathrm{base}_{b'}] + 0.15\,\mathbf{1}[\mathrm{carb}_b = \mathrm{carb}_{b'}] \\ & + 0.10\,\mathbf{1}[\mathrm{dairy}_b = \mathrm{dairy}_{b'}] + 0.05\,\mathbf{1}[\mathrm{alc}_b = \mathrm{alc}_{b'}] \\ & + 0.15\Big(1 - \tfrac{|\mathrm{caff}_b - \mathrm{caff}_{b'}|}{3}\Big) + 0.15\Big(1 - \tfrac{|\mathrm{sweet}_b - \mathrm{sweet}_{b'}|}{3}\Big). \end{aligned} \tag{17}$$

Both similarities are symmetric and lie in $[0, 1]$.

Definition 10.3 (Edge weight)**.**

$$w(b, b') = \alpha\ \mathrm{sim}_T + \beta\ \mathrm{sim} + \delta\ \mathrm{sim}_{\mathrm{emb}} + \eta\ \mathrm{sim}_{\mathrm{prof}} + \gamma\ \mathrm{sim}_{\mathrm{cop}}, \qquad \alpha + \beta + \delta + \eta + \gamma = 1, \tag{18}$$

every term evaluated at (b, b'), with the content similarity sim of (5) and the learned *co-preference* term sim_{cop}: the cosine of the two Bevs' vectors of z-scores over their common raters, 0 while fewer than 3 users have ranked both. Decided 2026-10-14: the first version uses $(\alpha, \beta, \delta, \eta, \gamma) = (0.25, 0.15, 0.30, 0.30, 0)$ and switches γ on (renormalising the others) once co-raters exist. Weights are computed over a candidate set, the union of same category, same brand and the 50 nearest by embedding cosine (`scripts/graph/build-edges.ts`); each term is rounded to six decimals before w is formed, so the stored components reproduce the stored weight exactly. A pair is kept when either endpoint ranks it among its $K_E = \text{EDGE_TOP_K} = 20$ heaviest with $w \geq \text{EDGE_MIN_WEIGHT} = 0.3$ (`topEdges`; ties on w go to the smaller id in code-point order, which for lowercase-hex uuids is also `uuid` byte order and `collate "C"`), so a popular Bev can have more than 20 incident edges. A pair with no stored edge falls back to sim_T alone (M-21).

Example 10.4 (Coca-Cola and Pepsi)**.** Same leaf (cola) gives $\text{sim}_T = 1$; content gives 0.5 (same category and price tier, different brands, no shared flavour tokens); two colas embed close together, sim_{emb} near 0.9; identical profiles give $\text{sim}_{\text{prof}} = 1$. Hence $w \approx 0.25 + 0.075 + 0.27 + 0.30 \approx 0.90$. For Coca-Cola and a plain sparkling water the tree, content and profile terms all fall (different leaf, no shared tokens, different base and sweetness), and w drops below 0.4. The content term alone would have put the two colas at 0.5, which is why the embedding and profile terms were added. These are the golden-vector figures (0.895 and 0.315, `edges.vectors.json`). The first real run against prod (2026-10-24, #121; 383 Bevs embedded with `gte-small`, 4669 edges stored) measured Coca-Cola–Pepsi $w = 0.9001$ and Coca-Cola–LaCroix Lime $w = 0.3809$, both inside the sanity bounds $w \geq 0.8$ and $w < 0.4$ that the job prints. With a flat taxonomy the second bound holds only while the embedding cosine of that pair stays at or below 0.88; the measured weight corresponds to a cosine of about 0.82.

Profiles for user-created Bevs (D-11, decided 2026-10-19). A Bev with no hand-labelled profile copies the profile of the nearest seeded Bev in the same category by embedding cosine (brand match, then id, as tie-breaks), marked *inferred* with its source and the cosine, provided the cosine is at least 0.6; below that no profile is written and the Bev's sim_{prof} term is 0, so it is weighed by the tree, content and embedding terms alone (`infer_bev_flavor_profile`, 20261023000200).

Remark 10.5 (Where the graph lives). Decided 2026-10-14, built in #115, #117, #118 and #121 (A-15, A-16 (1)). Postgres is the system of record for Bevs, users and rankings, and it is also where the weights are computed: the `embed` Edge Function writes e_b to `bev_embeddings` (pgvector, 384-d, on every Bev insert or update, plus a backfill), and the nightly job `scripts/graph/build-edges.ts` (04:50 UTC) reads Bevs, categories, profiles and embeddings, weighs the candidate pairs with `@bevi/ranking` and replaces `bev_edges` in one transaction, stamping `RANKING_VERSION`. No step of that pipeline touches Neo4j, so the ranking flow and the recommender read weights from Postgres and never wait on the graph. Neo4j AuraDB Free (`Bev`, `Category`, `Brand` and `Flavor` nodes; `SIMILAR_TO` edges carrying w and its components; pseudonymous `RANKED` edges; a vector index over e_b) is the mirror of `bev_edges`, where similarity is inspected and where the co-rating neighbourhoods behind sim_{cop} will be computed; the mirror sync (A-16 steps 6, 8 and 9) is a later increment, the instance sits on no request path, and the nightly `(:Meta)` write is what keeps a Free instance from pausing (`docs/graph.md`).

Anchored placement. Let $L = (\ell_0, \ldots, \ell_{n-1})$ be the user's bucket list and b the new Bev. Partition L into *tiers* by similarity to b: T_1 = same leaf category, T_2 = same parent, …, or more generally by decreasing $w(b, \ell_i)$ bands. Let $A = (a_1, \ldots, a_k)$ be the members of the most similar non-empty tier, in list order, at positions $p_1 < \cdots < p_k$ in L.

Algorithm 1 Anchored placement of b into L

if A is empty **then return** ordinary placement (Section 2) on L
end if
Phase 1. Run the placement state machine on the anchor list A (length k). ▷ every question is like-for-like
if a TIE against a_j ended Phase 1 **then return** $p_j + 1$
end if
Let $j^* \in \{0, \dots, k\}$ be the resulting slot among anchors.
$\text{lo} \leftarrow (j^* = 0 \ ? \ 0 : p_{j^*} + 1)$; $\text{hi} \leftarrow (j^* = k \ ? \ n : p_{j^*+1})$ ▷ candidate indices in L
Let $G = (\ell_{\text{lo}}, \dots, \ell_{\text{hi}-1})$ be the gap, $g = \text{hi} - \text{lo}$ Bevs, all non-anchors.
Phase 2. Run the placement state machine on G, giving slot $r \in \{0, \dots, g\}$. ▷ cross-category, only inside the gap
return $\text{lo} + r$

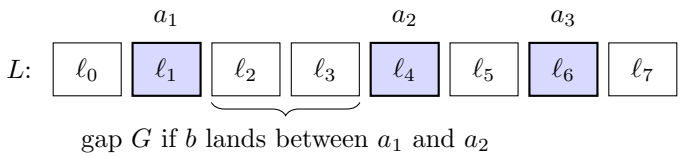


Figure 1: Anchors (shaded) are the user's ranked Bevs most similar to the new Bev. Phase 1 binary-searches the anchors only; Phase 2 searches the gap between the two anchors the new Bev landed between.

Theorem 10.6 (Properties of anchored placement)**.** *Against a consistent oracle with true slot t in L:*

(i) the returned index is t, and lies in $\{0, \dots, n\}$ for any sequence of answers;
(ii) every logged comparison agrees with the final index in the sense of Proposition 2.6;
(iii) at most $\lceil \log_2(k+1) \rceil + \lceil \log_2(g+1) \rceil$ questions are asked, the first $\lceil \log_2(k+1) \rceil$ of them against Bevs of the most similar tier; if the gap is empty no cross-category question is asked at all;
(iv) the first question is always against a Bev of the most similar non-empty tier.

Proof. (i) Consistency on L implies consistency on the subsequence A with true anchor slot j^* such that $p_{j^*} < t \le p_{j^*+1}$ (with the conventions $p_0 = -1$, $p_{k+1} = n$), so Phase 1 returns j^* by Theorem 2.5 and $t \in [\text{lo}, \text{hi}]$. The gap G is exactly $\ell_{\text{lo}}, \dots, \ell_{\text{hi}-1}$ and the oracle is consistent on it with true slot $t - \text{lo}$, so Phase 2 returns $r = t - \text{lo}$. For arbitrary answers, Phase 1 yields $j^* \in [0, k]$ hence $0 \le \text{lo} \le \text{hi} \le n$, and Phase 2 yields $r \in [0, g]$, so $\text{lo} + r \in [0, n]$. (ii) A Phase 1 answer against a_j constrains j^* relative to j, hence the final index relative to p_j in the same direction; a Phase 2 answer constrains r relative to a position inside the gap. (iii) and (iv) follow from Theorem 2.3 applied to each phase and from the construction of A. □

Remark 10.7 (Cost). In the worst case ($k \approx n/2$, a wide gap) anchored placement asks about twice as many questions as plain binary search; in the typical case the gap between two adjacent anchors is short and the total is close to $\lceil \log_2(n+1) \rceil$. The product requirement, that the like-for-like questions come first, is met exactly. The result keeps the existing contract (`insertIndex`, `expectedBucketSize`, `comparisons`), so `submit_ranking` is unchanged; the app needs each bucket Bev's category and brand (today `listBucket` returns ids only). An alternative, per-category lists with no cross-category questions at all, would give up the single comparative order behind (1); anchored placement is preferred for that reason.

10.2 Taste Profile

The session defined the Taste Profile as "your average score of cafe drinks could be this number out of 10" and "you prefer these kinds of drinks".

Definition 10.8 (Taste Profile)**.** For user u the profile is the table $\{(c, n_{u,c}, \mu_{u,c}, \mathrm{aff}_{u,c}) : n_{u,c} \geq 1\}$ ordered by $\mu_{u,c}$ descending. The display uses the raw means (what the user recognises); recommendations keep using aff (comparable across users). A preference statement "prefers c over c'" is shown when $n_{u,c}, n_{u,c'} \geq 3$ and $\mu_{u,c} - \mu_{u,c'} \geq 1$.

The table `user_category_stats` holds exactly this, written by `refresh_user_category_stats` after every ranking write and nightly for everyone (A-11, 20261019000100; means and aff stored to three decimals), and read through `get_taste_profile`, which returns the rows with $n_{u,c} \geq 1$ ordered by $\mu_{u,c}$ descending, then the category's sort order, then its id. The pgTAP suite checks the stored rows against the "five rankings" case of `user-stats.vectors.json`.

10.3 Bev of the day and Bev of the week

Definition 10.9 (Period winner)**.** Fix the zone America/New_York ("strictly New York City"). For a period P (a local calendar day, or an ISO week) let R_P be the rankings created in P and $B_P = \{b : \exists\, r \in R_P \text{ with Bev } b\}$ the Bevs ranked in P. The winner is

$$b^*_P = \operatorname*{arg\,max}_{b \in B_P} \big(\mathrm{bayes}_b,\ |R_P(b)|,\ n_b,\ -\mathrm{name}(b)\big) \tag{19}$$

in lexicographic order, then the Bev's id, where $|R_P(b)|$ is the number of rankings of b in P and bayes_b is the Bev's current Bayesian score (4) with the global prior. Candidates are the active Bevs the viewer may see (the age gate applies) and there is no MIN_RATERS gate, so one ranking today can win. Decided 2026-10-18 (D-02): if $B_P = \emptyset$, show the winner of the most recent non-empty period with its date, flagged as not current (`bev_of_period_at`, 20261018000100, which the pgTAP suite pins on a fixed clock).

Remark 10.10. Using the current bayes_b rather than only the period's scores keeps the card consistent with the leaderboard the user can open, and lets a single ranking today surface a Bev while the prior still tempers it. The alternative, a period-local shrunk mean $(Cm + \sum_{r \in R_P(b)} s_r)/(C + |R_P(b)|)$, rewards what was *said* today rather than what is *good*; it is one line to switch.

10.4 Likes and the Explore ranking

Definition 10.11 (Like count and scope)**.** $\ell_p \in \mathbb{N}$ is the number of distinct users who liked post p. For user u with follow set $F(u)$, the Explore scope is the second hop only, $E(u) = \big(\bigcup_{v \in F(u)} F(v)\big) \setminus (F(u) \cup \{u\})$: strangers whose posts the feed does not already show. Decided 2026-10-19 (D-01, D-15): while Bevi has fewer than 20 profiles the scope widens to every user outside $F(u) \cup \{u\}$, and Explore posts are read-only (no likes, counts or comments are shown).

Definition 10.12 (Hotness)**.** For a post p created at t_p, viewed at time t, with age $h = (t - t_p)$ in hours and gravity $\gamma_{\mathrm{hot}} = 1.5$,

$$\mathrm{hot}(p, t) = \frac{\ell_p + 1}{(h + 2)^{\gamma_{\mathrm{hot}}}}. \tag{20}$$

Explore lists the posts of $E(u)$ from the last 14 days by hot descending, then post id descending, with t the server's clock (never a caller parameter, so the like counts cannot be read off by moving it)

and h clamped at 0 for a future-dated post (`explore_hotness` and `get_explore`, 20261019000400). "Bevs getting the most likes" over a window W is $\mathrm{pop}_W(b) = \sum_{p \in W,\ \mathrm{bev}(p)=b} \mathrm{hot}(p,t)$; not built, since no screen asks for it yet.

Proposition 10.13. hot *is increasing in likes and decreasing in age; a post with ℓ likes outranks a post with ℓ' likes that is Δ hours younger iff* $(\ell+1)/(\ell'+1) > \big((h+2)/(h-\Delta+2)\big)^{\gamma_{\mathrm{hot}}}$. *With* $\gamma_{\mathrm{hot}} = 1.5$ *a day-old post needs about* 2.5 *times the likes (plus one) of a twelve-hour-old post to stay level.*

Proof. Direct from (20); the numeric example is $((24+2)/(12+2))^{1.5} \approx 2.53$. □

10.5 Smart Recommend on the map

Parked by the session's guardrail ("too many features") but specified so it can be built when wanted.

Definition 10.14 (Venue score)**.** A venue v carries a set of Bevs $M(v)$ (for chains, a shared assortment per chain plus venue-specific items). With category filter $F \subseteq C$ and untried set $\bar{R}(u) = B \setminus R(u)$,

$$\mathrm{score}(u, v; F) = \max\big\{\, \hat{s}_{u,b} : b \in M(v) \cap \bar{R}(u),\ \mathrm{cat}(b) \in F,\ \mathrm{conf}_{u,b} \geq 0.25 \,\big\}, \tag{21}$$

or $-\infty$ if the set is empty. Smart Recommend shows exactly the venues with $\mathrm{score}(u, v; F) \geq \theta$, with $\theta = 7$ (the LOVED threshold), and sorts them by score.

The maximum (rather than a mean) answers the question the user asks on a map, "is there something here I would love?", and inherits the lock rule of Section 6 so low-confidence guesses never light up a venue.

10.6 Nutrition normalisation

A vision model reads the label photo into per-serving facts $x = (x_1, \ldots, x_d)$ (calories, fat, sodium, carbohydrate, sugars, protein, caffeine, ...) together with the serving volume V in millilitres.

Definition 10.15 (Per 100 ml)**.** $x_i^{(100)} = 100\, x_i / V$. Per-container values multiply x_i by the number of servings per container.

Definition 10.16 (Acceptance checks)**.** An extraction is accepted when all values are non-negative, sugars $\leq$ carbohydrate, added sugars $\leq$ sugars, $V > 0$, and the energy identity holds within tolerance:

$$|\mathrm{calories} - (4\,\mathrm{carbohydrate} + 4\,\mathrm{protein} + 9\,\mathrm{fat})| \leq \max(10,\ 0.15 \cdot \mathrm{calories}).$$

Otherwise the result is stored as *needs review* and nothing is shown.

Nutrition is shown on the Bev's own page to whoever opens it, never in the feed or in rankings (the session: "just for the user themselves to see").

10.7 Duplicate detection and merging

Definition 10.17 (Duplicate score)**.** For a query q (name, brand, category, vessel) and an existing Bev b, with trigram similarity tri (`pg_trgm`) over the normalised "brand name" text and its aliases,

$$d(q,b) = \max\big(\mathrm{tri}(q,b), \max_{a \in \mathrm{aliases}(b)} \mathrm{tri}(q,a)\big) + 0.15\,\mathbf{1}[\mathrm{brand}(q) = \mathrm{brand}(b)] + 0.10\,\mathbf{1}[\mathrm{cat}(q) = \mathrm{cat}(b)]\,, \tag{22}$$

and $d(q,b) = 1$ whenever both carry the same UPC. "Did you mean…" shows the top five with $d \geq 0.3$ (the `pg_trgm` default); a user-flagged duplicate with $d \geq 0.6$ is queued for merge review, below that for manual review.

Definition 10.18 (Merge)**.** Merging b_{src} into b_{dst}: for each user who ranked both, keep the ranking with the better (bucket, position) and remove the other; re-point remaining rankings, aliases and Want to Try rows; recompute statistics; set b_{src} to status *merged* with `merged_into` $= b_{\mathrm{dst}}$.

Proposition 10.19. *A merge preserves, for every user, the relative order of all Bevs other than the merged pair, and every stored score remains inside its band.*

Proof. Removal of one ranking followed by re-spacing is a monotone relabelling of positions in that bucket (Proposition 3.2(ii)); other buckets are untouched; band membership is Proposition 3.2(i). □

Status: the "Did you mean…" step is live as `find_similar_bevs`, which uses the trigram part of (22) (the maximum over the Bev's text and its aliases) with the category as a filter rather than a bonus; the brand and category bonuses, the duplicate report and `merge_bev` are the S-13 plan.

Vessels (can, glass bottle, plastic bottle) are *not* distinct Bevs and must never enter the name: the natural key is (category, brand, name, flavour). A genuinely different product (Mexican Coca-Cola, cane sugar, glass) is its own Bev with its own icon.

10.8 One ranking per Bev per user

The ranking relation is a partial function $U \times B \rightharpoonup [0,10]$: a user has or has not ranked a Bev, and if so holds exactly one score. Re-ranking *updates* this value (and re-spaces the old and new buckets); it never appends. Consequently n_b counts people, not drinks, and "how many times you drank it" is deliberately not a quantity in this system (session decision; the database enforces it with a unique constraint on (user, Bev)).

11 Implementation notes, divergences and versioning

Implementation note 11.1 (Where the maths runs)*.* `@bevi/ranking` is pure TypeScript with no I/O or clock; Postgres holds *twins* of the formulas that touch stored data (re-spacing, statistics, Bayesian score, rank points, streaks) and the two are held equal by the golden vectors in `shared/fixtures` (one `*.vectors.json` file per formula), regenerated with `npm run vectors` on every `RANKING_VERSION` bump. Predictions and recommendations run in the client over the snapshot returned by `get_prediction_snapshot()` (decided 2026-10-12; revisit at about 200 users or a 1 MB snapshot).

Implementation note 11.2 (Prior mean)*.* `bayes_prior(p_category_id)` (20261019000200, #112) is the SQL twin of `globalPrior` and `categoryPrior`: a category's own mean at 30 rankings, else the global mean, and 5.0 on an empty system; `get_bev_leaderboard` uses it, so a Water on the water board now shrinks toward Water's mean. Still different: `get_feed` (20261014000300), `get_user_feed`

(20261018000300), `get_explore` (20261019000400) and `bev_of_period_at` (20261018000100) read `coalesce((select mean_score from global_stats), 0)`, so on a system with no rankings their prior is 0 rather than 5.0; invisible in production, where `global_stats.rating_count > 0`. The global branch everywhere reads `global_stats.mean_score`, a `numeric(5,3)` refreshed every 15 minutes, while `bayes_prior`'s category branch is live and exact. `refresh_global_stats` writes `bev_stats.bayes_score` with the global prior by design (one number per Bev), but updates only rows with `rating_count > 0`, so when a Bev's last ranking is removed the old figure stays in the column until the Bev is ranked again.

Implementation note 11.3 (Name order). `get_bev_leaderboard` has ordered by `bayes desc, rating_count desc, name collate "C", id` since 20261019000200, the order of Section 5; the earlier "Bayesian score only" divergence is closed, and `user_category_stats.affinity_z` has had a writer since 20261019000100 (A-11). One difference remains on the TypeScript side: `leaderboardOrder` compares names with JavaScript `<`, which orders UTF-16 code units, whereas `collate "C"` orders code points. The two disagree only when one name contains a character outside the Basic Multilingual Plane and the other a character in U+E000–U+FFFF at the same position (an emoji against a private-use or specials character). Every catalog name is ASCII, but `create_bev` does not restrict names. The fix is a code-point comparison in `bayesian.ts` (and the same `cmp` in `starter.ts`), with a `RANKING_VERSION` bump and a vector case that exercises such a pair.

Implementation note 11.4 (Streak history in SQL). `touch_streak` (20261011000700) recomputes D_u from `rankings.ranked_at`. A re-rank deletes the row and re-inserts it with `ranked_at = now()`, and a removal deletes it, so the original day leaves D_u whenever it held no other ranking; `feed_events` cascades on the same delete, and `comparisons` rows exist only when a question was asked. Section 9 counts ranking events, so after such a re-rank or removal the SQL current and longest streaks can be shorter than the specification's. Fix: an append-only ranking-event log (user, Bev, kind, timestamp) written by `submit_ranking` and `rerank_bev` and never deleted, which `touch_streak` reads; the `streakFromTimestamps` cases of `streak.vectors.json` are its test.

Implementation note 11.5 (Comparison log). `submit_ranking` and `rerank_bev` check insertIndex $\in$ $[0, \texttt{expectedBucketSize}]$ and the list length, and store every comparison with its tie flag, but do not re-check Proposition 2.6 against the submitted index. The log is a record, not yet a constraint: a client bug could store an inconsistent log without being rejected.

Implementation note 11.6 (Stored precision). Stored scores are exact (`numeric(4,2)` is cents/100). Aggregates are rounded on storage: `bev_stats.avg_score`, `score_stddev` and `bayes_score` to two decimals; `user_stats`, `user_category_stats` and `global_stats.mean_score` to three. `bayes_score` is computed from the exact `sum_score`, not from the rounded mean. The live leaderboard, feed and Explore figures are unrounded `numeric`; `bev_of_period_at` returns two decimals. The prediction snapshot computes user statistics live, so the client's z-scores are exact. The golden vectors are compared unrounded at 10^{-6} in TypeScript and at column scale in pgTAP.

Implementation note 11.7 (What pgTAP checks against the vectors). `08_taste_profile` pins one case of `user-stats.vectors.json`; `09_leaderboard_parity` pins `bayes_prior` and the full leaderboard order on its own fixture, including the `collate "C"` tail; `07_bev_of_period` pins Section 10.3 on a fixed clock; `10_explore` pins one value of (20). No SQL test runs `score-from-position.vectors.json` against `ranking_score`, `bev-stats.vectors.json` against `stddev_pop`, the `bayesianAverage` cases against `bayes_score`, `streak.vectors.json` against `touch_streak` (the timezone cases are where the engines could differ) or `rank-points.vectors.json` against `user_rank_points`. Those are the gaps to close in A-14's follow-up.

Implementation note 11.8 (Exploration slots). The development document lists the three slots for

a 20-item list as 4, 9, 15; the implemented even-spacing rule (15) gives 4, 9, 14. The code is the specification here.

Version	Date	Change
1.0.0	2026-10-09	First vectors: bands 7/4, placement, exact cents, statistics, Bayesian average, rank points, streaks
1.1.0	2026-10-11	Prediction snapshot, Layers 1–2, blend, exploration, starter set
1.2.0	2026-10-14	Bev-graph edge weights: tree (16), profile (17) and embedding similarity, w (18), top-K_E selection, as `edges.vectors.json` (M-24); earlier vectors unchanged apart from the header; current
(next)	—	Anchored placement (Section 10.1) when adopted; requires a bump and new vectors

A Constants

All values from `packages/ranking/src/constants.ts`, `RANKING_VERSION` 1.2.0.

Constant	Value	Role
BANDS	$[0, 4]$, $[4, 7]$, $[7, 10]$	Score bands for DISLIKED, FINE, LOVED; the first two exclude their top edge
SIGMA_FLOOR	1.0	Floor on any standard deviation used for z-scores
MIN_N_USER	3	Below this a user's z-scores use global statistics; predictions withheld
MIN_N_CAT	3	Below this `effectiveCategoryStats` falls back to the user's overall statistics; exported and in the vectors, but the product shows raw category means (Section 10.2)
GLOBAL_FALLBACK	$(5.0, 2.5)$	Only when the system has no rankings
K_AFF	2	Shrinkage of category affinity
K_G	3	Shrinkage of a Bev's normalised global score
BAYES_C	3	Prior weight of the Bayesian average (raise to 5 when median $n_b > 10$)
BAYES_CATEGORY_PRIOR_MIN	30	Rankings a category needs before its own mean is the prior
MIN_RATERS	2	Leaderboard eligibility
SIM_WEIGHTS	0.40 / 0.30 / 0.20 / 0.10	Category, brand, flavour tokens, price tier
SIM_MIN	0.1	Pairs below this are dropped
PRICE_TIER_SPAN	3	Largest $\lvert\text{tier} - \text{tier}'\rvert$ (tiers 1–4)
LAYER1_WEIGHTS	0.40 / 0.40 / 0.20	Affinity, content, global
CONF1_USER_N, CONF1_MASS	10, 4	Layer 1 confidence saturation
CF_K	10	Neighbours per prediction
CF_MIN_COMMON	2	Minimum common items for a non-zero user similarity
CF_SIGNIFICANCE	5	Significance weighting of user similarity
LAMBDA_MAX, LAMBDA_USER_N	0.7, 15	Blend weight ceiling and user-history saturation
CONFIDENCE_LOCK	0.25	Below this the UI shows a lock
EPSILON	0.15	Share of the list given to exploration
JITTER_SD	0.15	Ordering-only jitter (z-units)
RECOMMEND_N	20	Default list length (API caps at 50)
STARTER_K, STARTER_CORE	10, 6	Onboarding set size and shared core
STARTER_MAX_PER_CATEGORY, STARTER_MIN_CATEGORIES	2, 5	Spread of the onboarding set
RANK_POINTS_WEIGHTS	1 / 3 / 2	Bevs ranked, current streak, adopted Bevs created
STREAK_PERIOD	day	Daily streak (weekly is a constant change)
EDGE_WEIGHTS	0.25 / 0.15 / 0.30 / 0.30 / 0	Tree, content, embedding, profile, co-preference (18)
EDGE_TOP_K, EDGE_MIN_WEIGHT	20, 0.3	Edges kept per Bev (K_E) and their minimum weight
PROFILE_WEIGHTS	0.40 / 0.15 / 0.10 / 0.05 / 0.15 / 0.15	Base, carbonation, dairy, alcohol, caffeine, sweetness (17)
PROFILE_LEVEL_SPAN	3	Largest caffeine or sweetness difference

B Symbols

Symbol	Meaning
U, B, C	Users, Bevs, categories; $\mathrm{cat}(b)$ the category of b
$R(u), R^{-1}(b)$	Bevs ranked by u; users who ranked b
$n_u, n_b, n_{u,c}$	Counts of rankings by user, of a Bev, by user within a category
$s_{u,b}$	Stored score of u for b (two decimals)
$\sigma(i, n)$	Score of position i in a bucket of n (eq. 1)
μ_u^*, σ_u^*	Effective mean and (floored) standard deviation of u
$z_{u,b}$	Normalised score of u for b
$\mathrm{aff}_{u,c}$	Category affinity (eq. 2); g_b normalised global score (eq. 3)
bayes_b	Bayesian leaderboard score (eq. 4); m its prior mean, C its weight
$\mathrm{sim}(b, b')$	Content similarity; $\mathrm{sim}'(u, v)$ significance-weighted user similarity
$\hat{z}^{(1)}, \hat{z}^{(2)}, \hat{z}$	Layer 1, Layer 2 and blended predictions in z-units; $\hat{s}$ in score units
$\lambda, \mathrm{supp}, \mathrm{conf}$	Blend weight, neighbour support, confidence
sim_T, w	Tree similarity and Bev-graph edge weight (Section 10.1)
A, G, k, g	Anchors, gap, and their sizes in anchored placement
$\mathrm{hot}(p, t)$	Time-decayed popularity of a post (eq. 20)